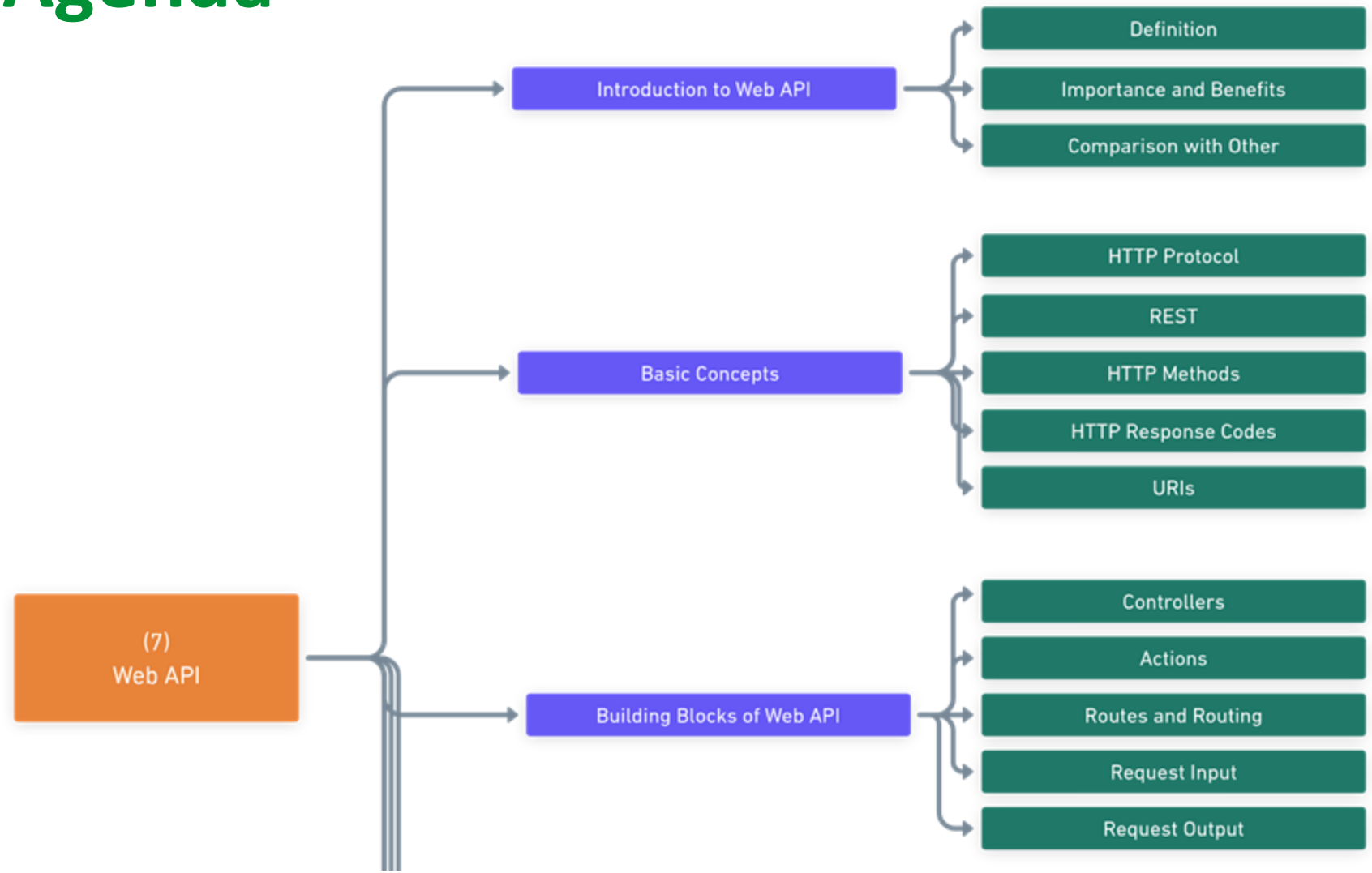
WEB API

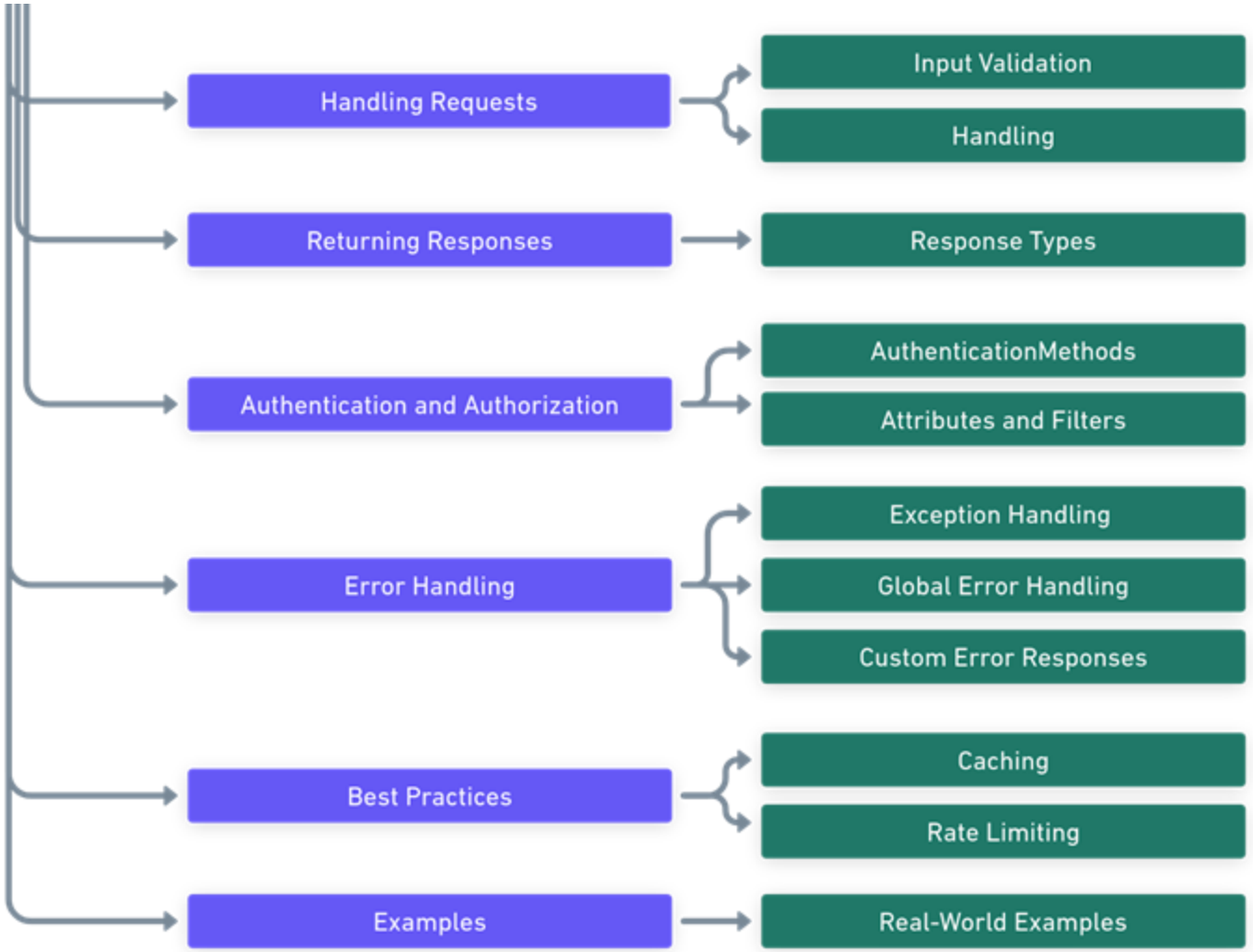
Tehnici avansate de programare

Nicoleta Scrimint

Agenda


(7)
Web API
Introduction to Web API
Definition
Importance and Benefits
Comparison with Other
Basic Concepts
HTTP Protocol
REST
HTTP Methods
HTTP Response Codes
URIs
Building Blocks of Web API
Controllers
Actions
Routes and Routing
Request Input
Request Output

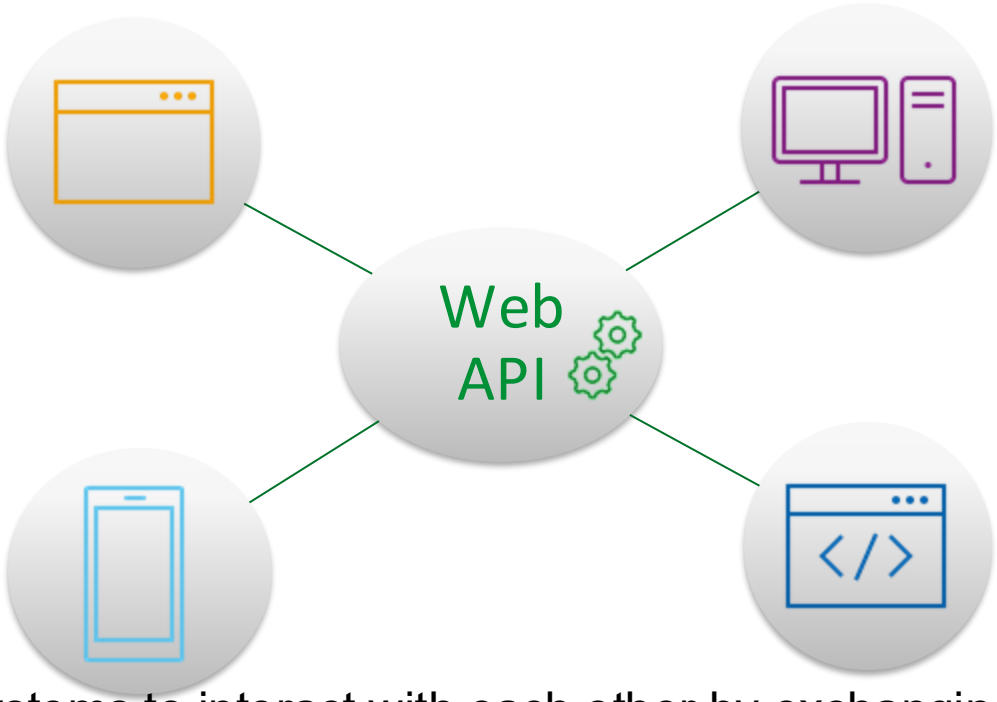
Agenda


Handling Requests
Input Validation
Handling
Returning Responses
Response Types
Authentication and Authorization
AuthenticationMethods
Attributes and Filters
Error Handling
Exception Handling
Global Error Handling
Custom Error Responses
Best Practices
Caching
Rate Limiting
Examples
Real-World Examples

INTRODUCTION TO WEB API

Definition of Web API

- **Web API** (Application Programming Interface) allows communication between different software systems over the internet.



It enables systems to interact with each other by exchanging data in a standardized format, typically using HTTP.

Importance and Benefits

Facilitates building **scalable** and **flexible** applications.

Enables **integration** with various platforms and devices.

Supports **cross-platform** development.

Promotes a **decoupled architecture**, allowing backend and frontend teams to work independently.

BASIC CONCEPTS

HTTP Protocol

- **HTTP** (Hypertext Transfer Protocol) is a protocol used for communication between clients and web servers.

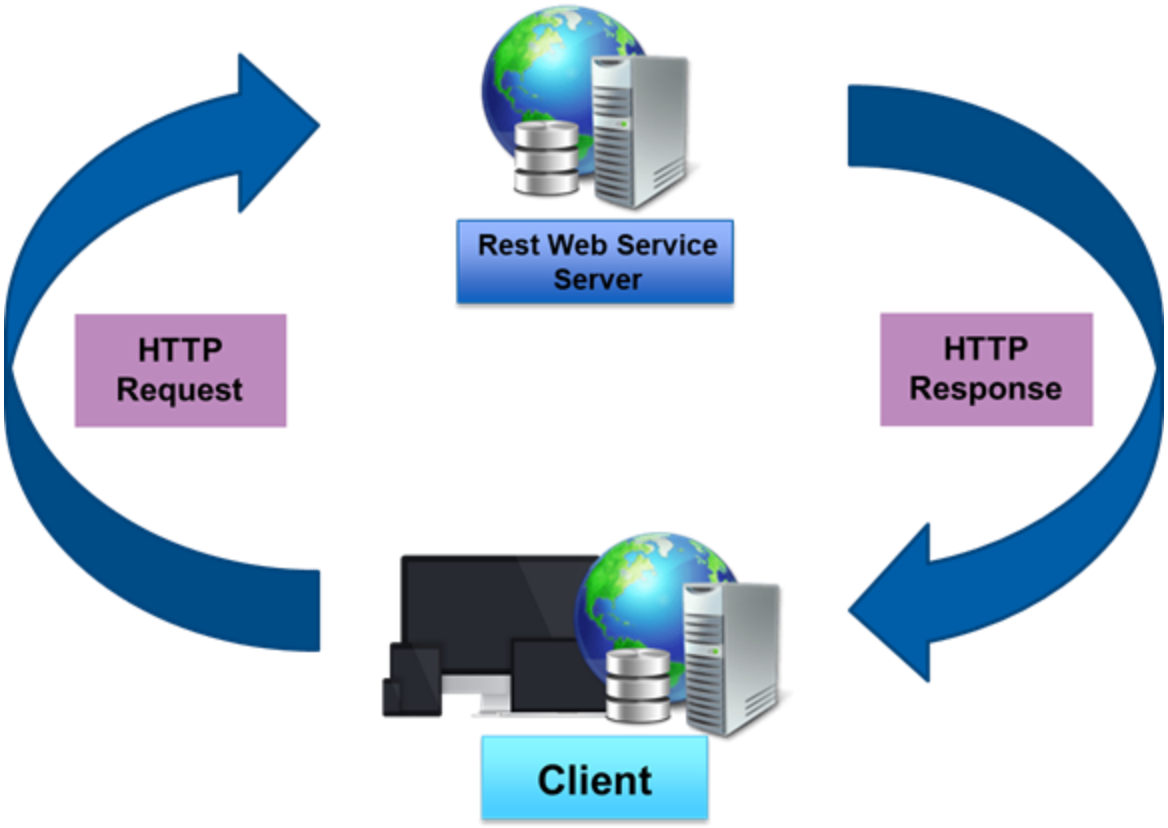
- HTTP follows a request-response model, where a client sends a request to a server, and the server responds to that request.

HTTP Protocol

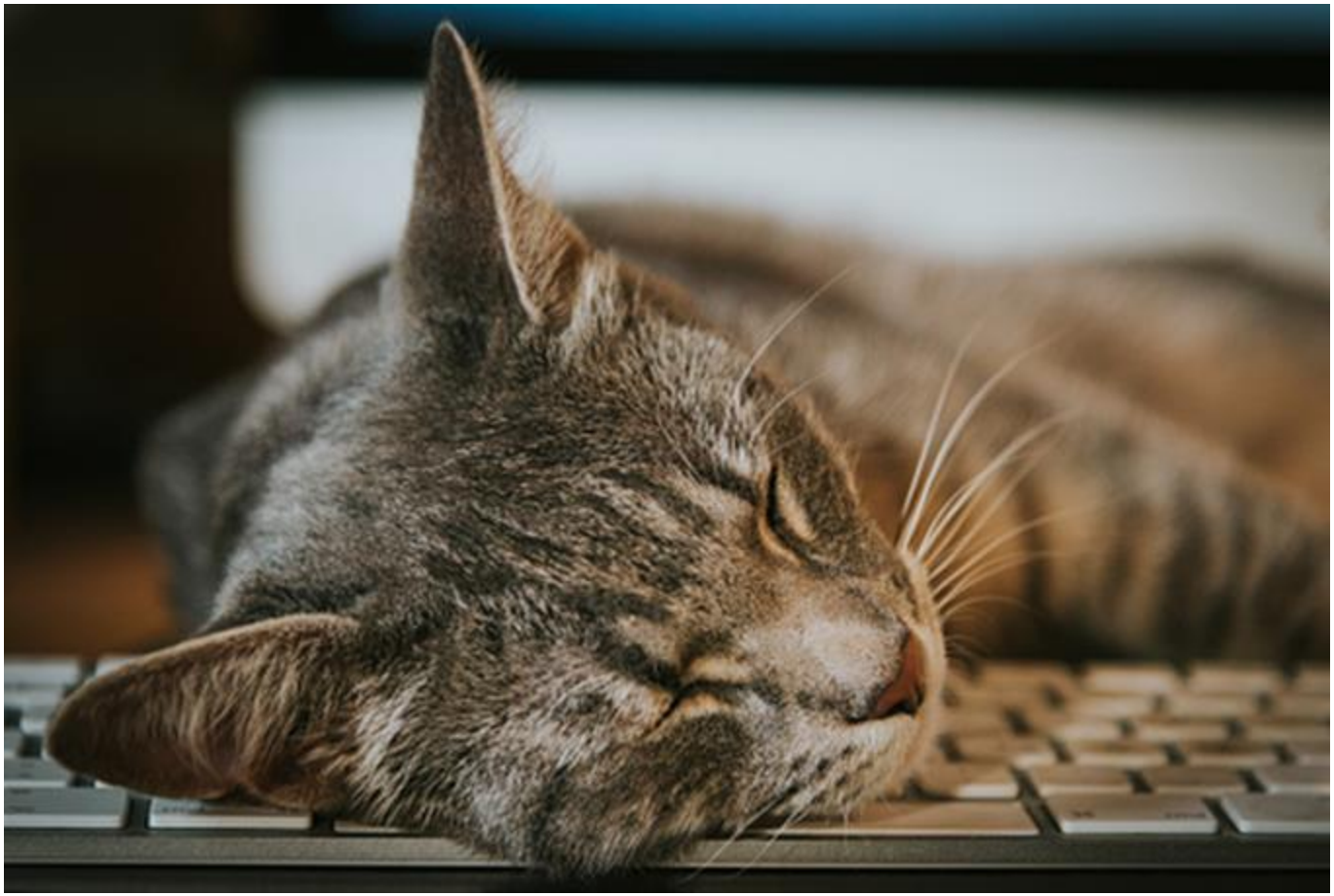
Here's a breakdown of the **request-response cycle**:

- Client Sends Request
- Server Processes Request
- Server Sends Response
- Client Receives Response
- Connection Closure

HTTP Protocol



RESTful Principles



RESTful Principles

RESTful Principles

RESTful architecture is a way to design web services that are **easy to use**, **flexible**, and **reliable**. It's based on a few key ideas:

- **Resource-based Design**: Everything in a RESTful system is treated as a resource, like a webpage, an image, or a user profile. Each resource has a unique address called a URI.
- **Statelessness**: Each request from a client to a server must contain all the information needed for the server to understand it. The server doesn't remember anything about previous requests from the same client. This makes the system simpler and more scalable.

RESTful Principles

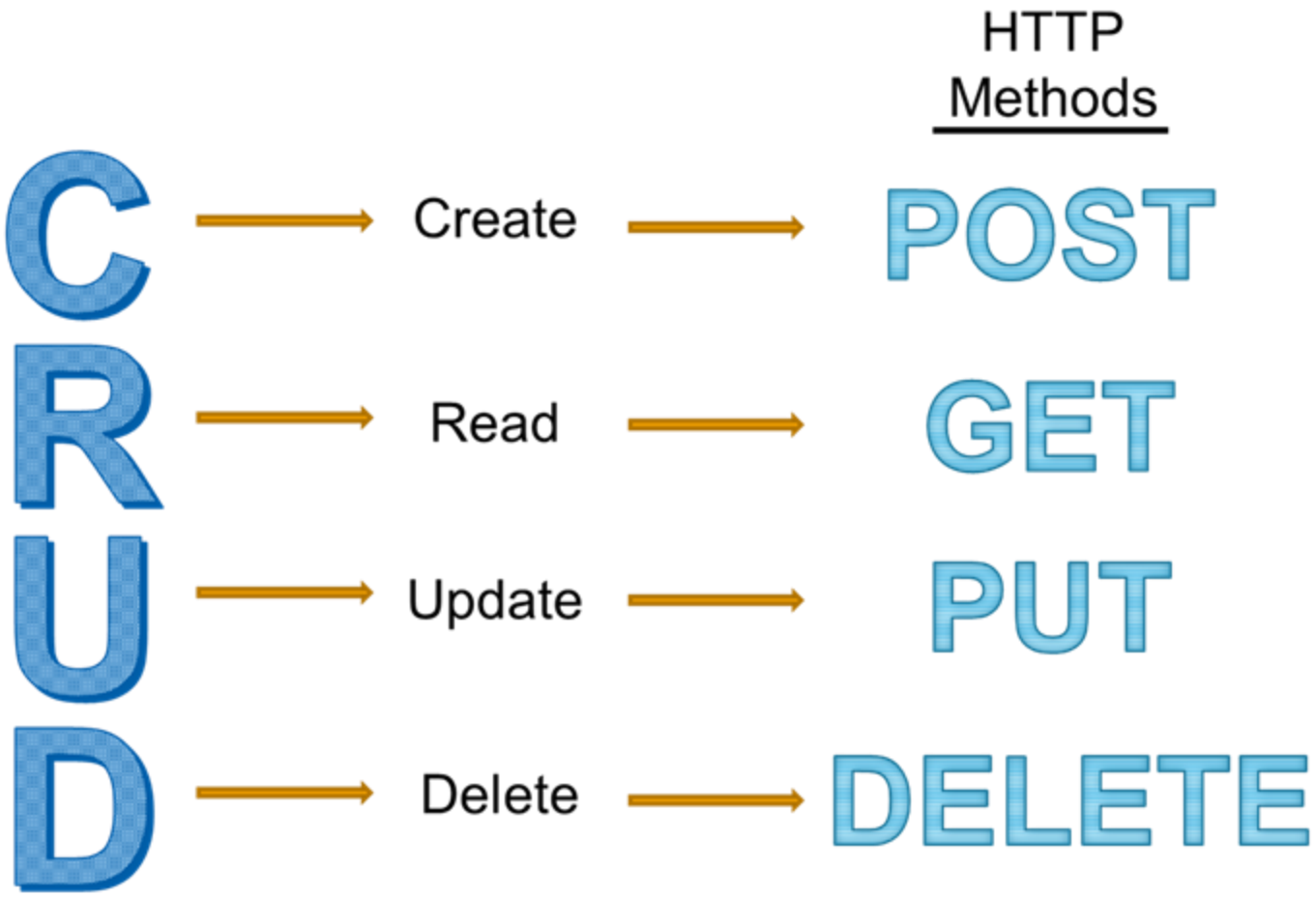
- **Uniform Interface**: RESTful systems use a set of standard rules for communication between clients and servers. This includes using HTTP methods like GET (to get data), POST (to send data), PUT (to update data), and DELETE (to delete data). It also means using standard data formats, like JSON or XML.
- **Layered System**: RESTful systems are organized into layers, with each layer having a specific role. This helps keep the system flexible and easy to manage.

By following these principles and using HTTP methods correctly, developers can create web services that are easy to understand, work well, and can be easily expanded and maintained over time.

HTTP Methods

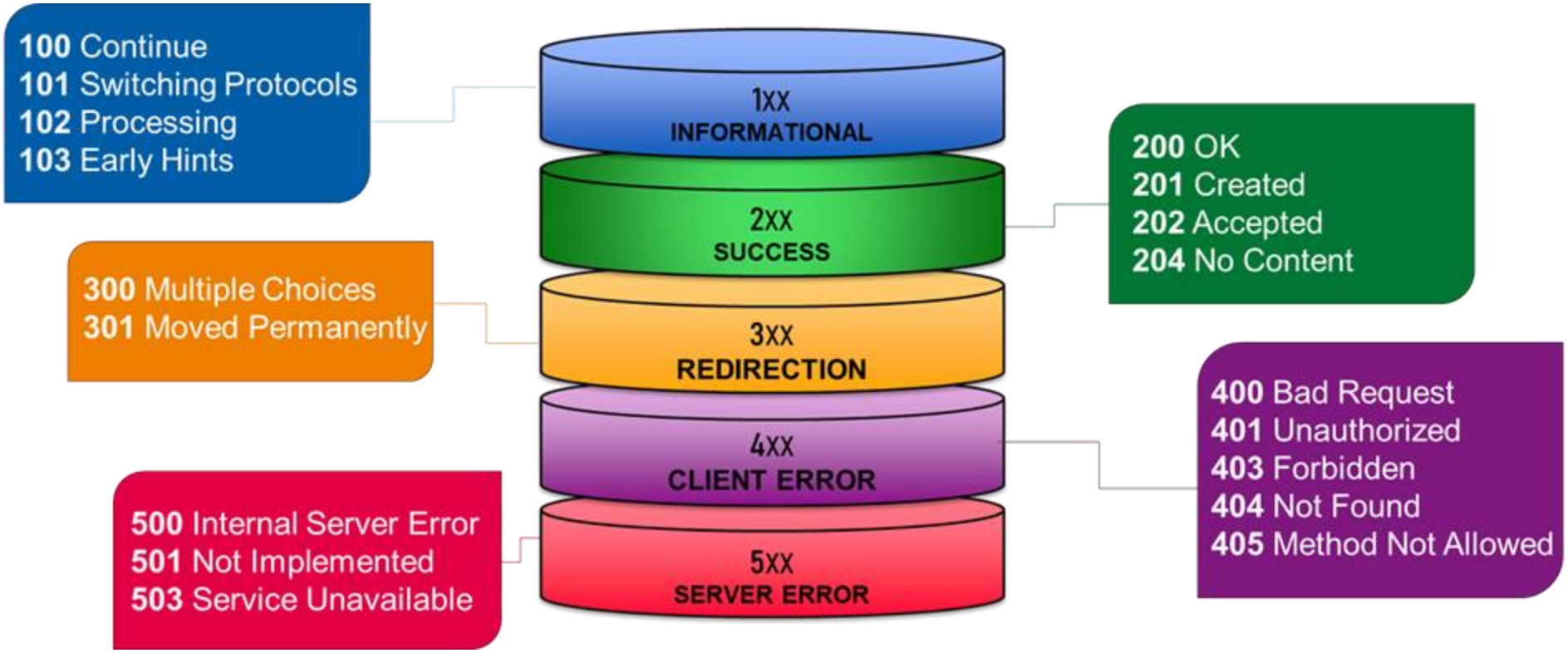
Method	Path	Action
GET	/objects	*Search for objects*
GET	/objects/{id}	*Get a specific object*
POST	/objects	*Create a new object*
PUT	/objects/{id}	*Update an object*
DELETE	/objects/{id}	*Delete an object*

HTTP Methods



HTTP Response codes

HTTP Response codes



URIs

URI (Uniform Resource Identifier)

Definition:

- A URI is a string of characters used to identify a resource. It can be a name or a locator, providing a way to uniquely identify a resource.

Purpose:

- URIs are used to specify the location or identifier of resources on the internet or within a system.

Example:

- **https://www.example.com/resource123** is a URI that identifies a resource located at the specified web address.

URLs

URL (Uniform Resource Locator):

Definition:

- A URL is a specific type of URI that specifies the location of a resource and the protocol used to access it.

Purpose:

- URLs provide a standardized way to reference resources on the web, including webpages, images, documents, and other files.

Components:

https://www.example.com:8080/path/to/resource?query_param1=value1#fragment_identifier

Scheme *Authority* *Path* *Query Parameters* *Fragment Identifier*

URNs

URN (Uniform Resource Name):

Definition:

- A URN is a specific type of URI that provides a persistent, location-independent identifier for a resource.

Purpose:

- URNs are used to uniquely identify resources without specifying their location or access method, making them suitable for long-term referencing.

Example:

urn:icnp:195010255

BUILDING BLOCKS OF WEB API

Controllers

- A **Controller** in Web API acts as a container for a group of related actions.

- It listens for incoming HTTP requests and dispatches them to the appropriate action methods based on the URL and HTTP method (GET, POST, PUT, DELETE, etc.).

The controller is a class that extends from ControllerBase (for APIs) or Controller (for MVC applications that include views).

Controllers

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        // Logic to retrieve and return users
    }

    [HttpPost]
    public IActionResult Create([FromBody] User user)
    {
        // Logic to create a new user
    }
}
```

Actions

- **Actions** are public methods within a controller class that correspond to individual operations a client can perform by sending a request to the server.
- Each action method is usually associated with a specific HTTP method through attributes like:
  - [HttpGet]
  - [HttpPost]
  - [HttpPut]
  - [HttpDelete]

These attributes help the framework route requests to the correct action method.

Actions

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        // Logic to retrieve and return users
    }

    [HttpPost]
    public IActionResult Create([FromBody] User user)
    {
        // Logic to create a new user
    }
}
```

Routes and Routing Attributes

- **Routing** is the mechanism that .NET uses to decide which controller and action to invoke in response to an incoming HTTP request.

- This decision is based on the URL pattern and HTTP method of the request, and the route configuration defined in the application startup.

- Controllers and actions can also use route parameters to capture data from the URL, making it available to action methods for processing.

Routes and Routing Attributes

```
[ApiController]
[Route("api/users")] // Route prefix for all actions in this controller
public class UserController : ControllerBase
{
    // Action for getting all users
    [HttpGet] // Matches GET requests to api/users
    public IActionResult GetAllUsers()
    {
        // Code to retrieve all users
        return Ok(users);
    }
```

Routes and Routing Attributes

```
// Action for getting a single user by ID
[HttpGet("{id}")] // Matches GET requests to api/users/{id}
public IActionResult GetUser(int id)
{
    // Code to retrieve a single user by id
    var user = users.FirstOrDefault(u => u.Id == id);
    if (user == null)
    {
        return NotFound();
    }

    return Ok(user);
}
```

Routes and Routing Attributes

```
[HttpPost("create-new")] // Custom route defined for this action
public IActionResult CreateProduct([FromBody] Product product)
{
    // Code to add the new product
    // Assuming you have a service that handles the creation logic
    var createdProduct = productService.CreateProduct(product);

    // The route name "GetProduct" would be the name of your HttpGet action
    return CreatedAtAction(nameof(GetProduct), new { id = createdProduct.Id }
}
```

Routes and Routing Attributes

```
app.UseHttpsRedirection();
app.UseAuthorization();

// Conventional routing
app.MapControllerRoute(
    name: "defaultApi",
    pattern: "api/{controller}/{action}/{id?}"
);

// Fallback to attribute routing for everything else
app.MapControllers();

app.Run();
```

Request Input: Model Binding

- **Model Binding** is the process where .NET takes incoming HTTP request data and creates objects that match the expected data types in your action methods.

This process is automatic, and the framework handles the conversion for you.

- For example, if your API needs to accept JSON data to create a new Product, you'll define a Product class, and then your action method will accept a Product type parameter.

Request Input: Model Binding

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}

[HttpPost]
public IActionResult CreateProduct([FromBody] Product product)
{
    // Add product to the database
    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}
```

Request Input: Model Binding

```
// Action for getting a single user by ID
[HttpGet("{id}")] // Matches GET requests to api/users/{id}
public IActionResult GetUser(int id)
{
    // Code to retrieve a single user by id
    var user = users.FirstOrDefault(u => u.Id == id);
    if (user == null)
    {
        return NotFound();
    }

    return Ok(user);
}
```

Request Output: Action Results

- **Action Results** are the way .NET API methods send responses back to the client. The framework provides several helper methods to return different HTTP status codes and data.

Example:

- Ok() sends a 200 OK response with optional data.
- CreatedAtAction() sends a 201 Created response
- NotFound() sends a 404 Not Found response.
- BadRequest() sends a 400 Bad Request response.

Request Output: Action Results

```
[HttpGet("{id}")]
public IActionResult GetProduct(int id)
{
    var product = /* code to fetch product by id */;
    if (product == null)
    {
        return NotFound();
    }

    return Ok(product);
}
```

HANDLING REQUESTS

Input Validation

Importance of validating input data:


Preventing Injection Attacks


Avoiding Bad Data


Ensuring Business Rules


Reducing Errors


Preventing Denial of Service (DoS) Attacks(overwhelm the system)

Input Validation

```
public class RegisterUserModel
{
    [Required(ErrorMessage = "Username is required.")]
    [MinLength(5, ErrorMessage = "Username must be at least 5 characters long.")]
    public string Username { get; set; }

    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Invalid Email Address.")]
    public string Email { get; set; }

    [Required]
    [StringLength(100, MinimumLength = 6, ErrorMessage = "Password must be between 6
    public string Password { get; set; }

    // Custom validation
    [AgeValidation(18, ErrorMessage = "You must be at least 18 years old.")]
    public int Age { get; set; }
}
```

Input Validation

```
public class AgeValidationAttribute : ValidationAttribute
{
    private readonly int _minimumAge;

    public AgeValidationAttribute(int minimumAge)
    {
        _minimumAge = minimumAge;
    }

    protected override ValidationResult IsValid(object value, ValidationContext vali
    {
        if (value is int age && age >= _minimumAge)
        {
            return ValidationResult.Success;
        }
        else
        {
            return new ValidationResult($"You must be at least {_minimumAge} years o
        }
    }
}
```

Input Validation

```
public class UsersController : ControllerBase
{
    [HttpPost("register")]
    public IActionResult Register([FromBody] RegisterUserModel model)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        // Proceed with registration logic
        // ...

        return Ok("User registered successfully");
    }
}
```

Input Validation

```
using FluentValidation;

public class ProductValidator : AbstractValidator<Product>
{
    public ProductValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("The product name is required.")
            .Length(2, 100).WithMessage("The product name must be between 2 and 100 c

        RuleFor(x => x.Price)
            .GreaterThan(0).WithMessage("The product price must be greater than 0.");
    }
}

public class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

```
builder.Services.AddControllers();
builder.Services.AddFluentValidation(fv => fv.RegisterValidatorsFromAssemblyContaining<ProductValidator>());
```

Handling

- **Content negotiation** is a process that takes place between the client and the server to determine the format of the data exchange.
- It allows clients to request data in a preferred format, and the server attempts to serve the data in that format if it can.

In the context of a Web API in .NET, this typically revolves around formats like JSON, XML, or others.

Handling

How content negotiation works in a Web API:

I. Accept Header
(E.g., Accept: application/json)

I. Server Evaluation

I. Serialization

I. Response
(E.g., Content-Type: application/json)

Handling

- .NET Web API supports content negotiation out of the box with built-in support for JSON, and it can be extended to support XML and *other formats*.

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers()
    .AddXmlSerializerFormatters(); // Add support for XML

var app = builder.Build();
```

Response Types

IActionResult

- IActionResult is an interface that various action result types implement.
  - It provides a way to return different HTTP responses (e.g., Ok, BadRequest, NotFound).

- When to Use:
  - Use IActionResult when you need the flexibility to return different types of responses from your API actions.
  - It's ideal for **CRUD** operations where the outcome might vary (success, failure, validation errors, etc.).

Response Types

```
[HttpGet("{id}")]
public IActionResult GetProduct(int id)
{
    var product = FindProduct(id);
    if (product == null) return NotFound();
    return Ok(product);
}
```

Response Types

JSONResult

- JSONResult is a specific action result type that returns JSON-formatted data to the client.
  - It explicitly sets the response content type to application/json.

- When to Use:
  - Use JSONResult when you explicitly want to return JSON-formatted data from an action.
  - It's useful when you want to ensure the client receives JSON, regardless of the Accept header sent in the request.

Response Types

```
public JSONResult GetJsonData()
{
    var data = new { Name = "Gadget", Price = 19.95 };
    return new JSONResult(data);
}
```

AUTHENTICATION AND AUTHORIZATION

Authentication Methods

- Authentication in a Web API is critical for determining the **identity** of users and **securing access**.

Common authentication methods:


JWT (JSON Web Tokens)


Oauth

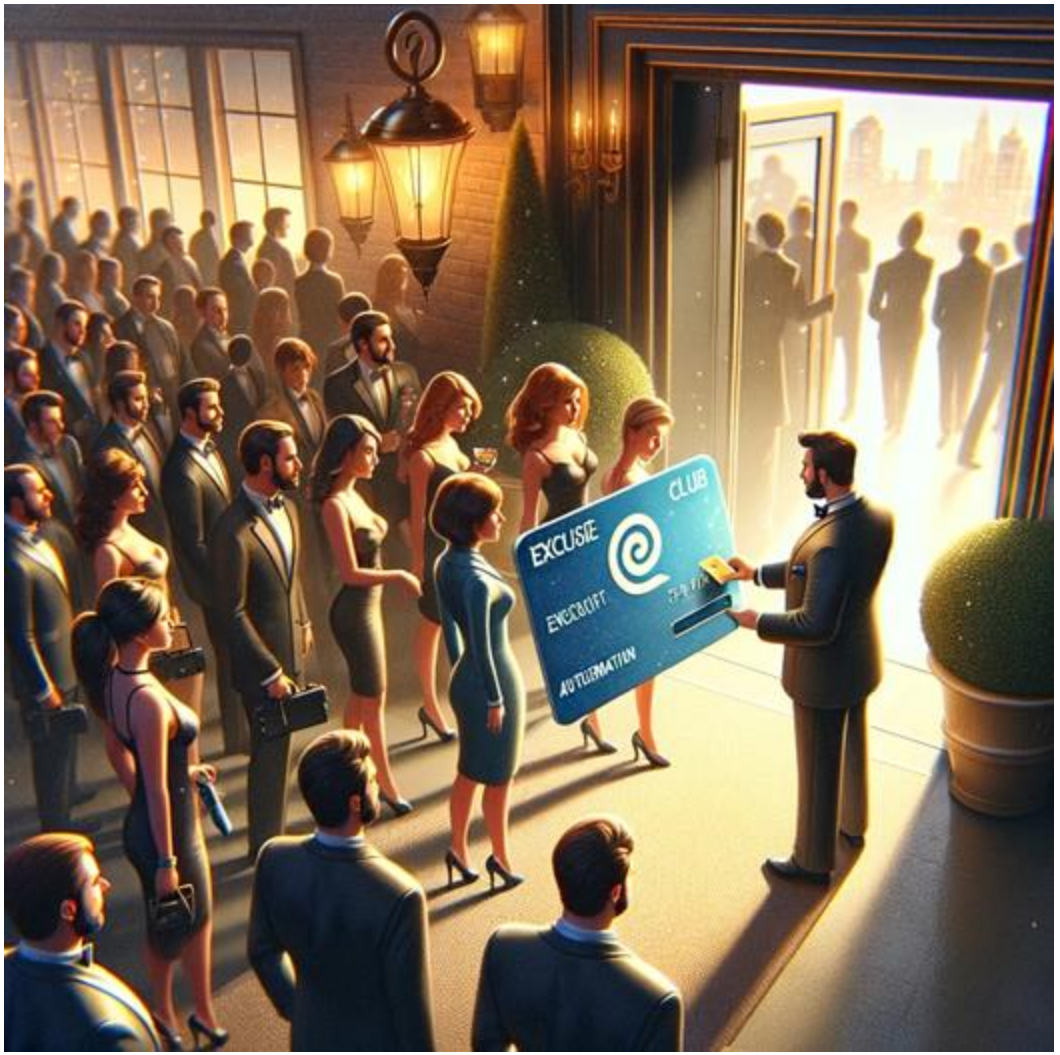

Basic Authentication

Authentication Methods

- JWT (JSON Web Tokens)

Authentication Methods

- Oauth



Authentication Methods

- Basic Authentication

Authorization Attributes and Filters

- **Authorization Attributes** are special markers that can be applied to controllers or actions to specify the authorization policy that should be applied.
  - The most common attribute is [Authorize], which can be used to indicate that a user must be authenticated to access the resource.
  - It can also be extended to require users to belong to specific roles or policies.

- **Authorization Filter** is a type of filter that runs before the action method is called.
  - It checks if the user has permission to perform the action requested.
  - If not, it can prevent the action from being executed.
  - Custom authorization filters can be created by implementing the IAuthorizationFilter or IAsyncAuthorizationFilter interface.

Authorization Filters and Attributes

```
[ApiController]
[Route("api/[controller]")]
public class SecureDataController : ControllerBase
{
    // This action can only be accessed by authenticated users
    [Authorize]
    [HttpGet]
    public IActionResult GetSecureData()
    {
        return Ok("This is secure data.");
    }

    // This action can only be accessed by users in the "Admin" role
    [Authorize(Roles = "Admin")]
    [HttpGet("admin")]
    public IActionResult GetAdminData()
    {
        return Ok("This data is only for admins.");
    }
}
```

ERROR HANDLING

Exception Handling

- **Exception Handling** refers to the mechanisms you put in place to deal with runtime errors so that your program doesn't crash.

```
[HttpGet("{id}")]
public ActionResult<Item> Get(int id)
{
    try
    {
        var item = repository.GetItem(id);
        if (item == null)
        {
            return NotFound();
        }
        return item;
    }
    catch (Exception ex)
    {
        // Log the exception
        return StatusCode(500, "An error occurred while pr
    }
}
```

It involves using try-catch blocks to gracefully handle errors that may occur during the execution of your application

Global Error Handling

- You can implement **global error handling** using built-in middleware to catch exceptions thrown from anywhere in your application.

```
app.UseExceptionHandler(appError =>
{
    appError.Run(async context =>
    {
        context.Response.StatusCode = StatusCodes.Status500InternalServerError;
        context.Response.ContentType = "application/json";

        var contextFeature = context.Features.Get<IExceptionHandlerFeature>();
        if (contextFeature != null)
        {
            // Log the exception
            await context.Response.WriteAsync(new ErrorDetails
            {
                StatusCode = context.Response.StatusCode,
                Message = "Internal Server Error. Please try again later."
            }.ToString());
        }
    });
});
```

Custom Error Responses

- **Custom error responses** allow you to provide more specific feedback to API consumers, such as different messages or formats based on the type of exception.

```
var (statusCode, message) = exception switch
{
    MyNotFoundException => (StatusCodes.Status404NotFound, "Resource not found."),
    MyUnauthorizedException => (StatusCodes.Status401Unauthorized, "Unauthorized acces
    MyValidationException ex => (StatusCodes.Status400BadRequest, ex.Message),
    _ => (StatusCodes.Status500InternalServerError, "Internal Server Error. Please try
};

context.Response.StatusCode = statusCode;
// Log the exception here using your logging mechanism

await context.Response.WriteAsync(new ErrorDetails
{
    StatusCode = statusCode,
    Message = message
}.ToString());
```

BEST PRACTICES AND PERFORMANCE OPTIMIZATION

Caching Strategies

- **Caching** is a technique that stores copies of data in a cache, or temporary storage area, so that future requests for that data can be served faster.

- In the context of a web API, caching can significantly improve performance by reducing database load and speeding up response times.

Caching Strategies

Caching Strategies

- Caching mechanisms:

In-Memory Caching

Distributed Caching

Response Caching

Data Caching

Caching Strategies

- **In-Memory Caching**

  - This stores data directly in the memory of the web server. It's fast but limited to the memory available on the server and is not shared across multiple servers.

```
builder.Services.AddMemoryCache();
```

```
private readonly IMemoryCache _cache;

public WeatherForecastController(IMemoryCache cache)
{
    _cache = cache;
}

[HttpGet]
public IActionResult Get()
{
    const string cacheKey = "weatherForecast";
    if (!_cache.TryGetValue(cacheKey, out List<WeatherForecast> forecast))
    {
        // The cache key doesn't exist, so get fresh data
        forecast = GetWeatherForecastData();

        // Set cache options
        var cacheEntryOptions = new MemoryCacheEntryOptions()
            .SetSlidingExpiration(TimeSpan.FromMinutes(1)); // Cache data f

        // Set the data in the cache
        _cache.Set(cacheKey, forecast, cacheEntryOptions);
    }

    return Ok(forecast);
}
```

Caching Strategies

- **Distributed Caching**
  - This involves a separate data store that can be shared across multiple servers in a web farm (e.g. Redis). This is ideal for applications that are scaled out across multiple servers.

Caching Strategies

- **Response Caching**
  - This refers to caching the response of an HTTP request. It can be done at various levels including the client-side (browser caching), server-side, or even by intermediate proxies/caches (like CDN caches).

Caching Strategies

- **Data Caching**
  - Instead of caching the entire response, you cache the data used to generate the response. This can be useful when different responses rely on the same underlying data.

Rate Limiting

- **Rate Limiting** is a technique used to control the number of incoming requests to a web server within a certain timestamp.
- This is crucial for preventing abuse, managing server load, and ensuring fair use of resources among users.

```
Install-Package AspNetCoreRateLimit
```

```
app.UseIpRateLimiting(); // Apply IP rate limiting
app.UseClientRateLimiting(); // Apply Client ID rate limiting
```

```
"ClientRateLimitPolicies": {
  "ClientRules": [
    {
      "ClientId": "special-client",
      "Rules": [
        {
          "Endpoint": "*",
          "Period": "1m",
          "Limit": 30
        }
      ]
    }
  ]
}
```

Rate Limiting

EXAMPLES

Real-World Examples

E-commerce

- Product Catalogs and Inventory Management
- Payment Processing
- Shipping and Logistics

Finance

- Banking Services
- Investment and Trading Platforms
- Credit and Loan Services